Experiment No. 3

Sampling Techniques Using GNU Radio

Anil Kumar Senapati

Aim

To study sampling techniques using GNU Radio and analyze the effects of proper sampling, undersampling, and oversampling in the frequency domain.

Objective

The objective of this experiment is to demonstrate the Nyquist Sampling Theorem and observe the effects of different sampling conditions using GNU Radio. The experiment analyzes spectral behavior under proper sampling, undersampling, and oversampling conditions.

Theory

Sampling is the process of converting a continuous-time signal into a discrete-time signal by taking samples at regular intervals.

According to the Nyquist Sampling Theorem, a signal can be reconstructed without distortion if the sampling frequency is at least twice the highest frequency component present in the signal.

If the sampling frequency is lower than the Nyquist rate, aliasing occurs, causing overlapping of spectral components and incorrect reconstruction of the original signal.

Oversampling employs a sampling frequency significantly greater than the Nyquist rate, resulting in improved spectral representation and reduced distortion.

Mathematical Background

According to the Nyquist Sampling Theorem,

$$f_s \geq 2f_m$$

where

- f_s = Sampling Frequency
- f_m = Maximum Frequency Component

Aliasing occurs when

$$f_s < 2f_m$$

The aliased frequency is given by

$$f_a = |f - nf_s|$$

where

- f_a = Aliased Frequency
- f = Original Frequency
- f_s = Sampling Frequency
- n = Integer

Equipment Required

- Computer/Laptop
- GNU Radio Companion (GRC)

Software Required

- GNU Radio
- Python (Installed with GNU Radio)

Block Diagram

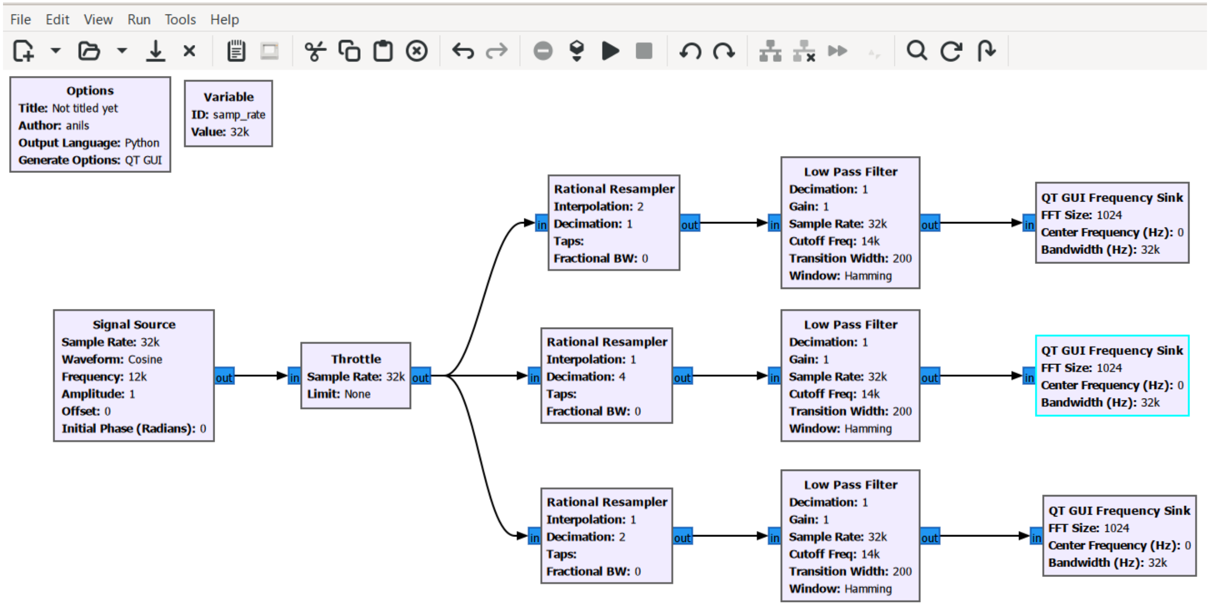

Figure 1: Sampling Techniques Flowgraph

GNU Radio Blocks Used

Block	Purpose
Signal Source	Generates cosine signal
Throttle	Controls processing speed
Rational Resampler	Performs interpolation and decimation
Low Pass Filter	Removes unwanted frequency components
QT GUI Frequency Sink	Displays frequency spectrum

Table 1: GNU Radio Blocks Used

Flowgraph Description

The flowgraph begins with a Signal Source block generating a cosine wave of frequency 12 kHz sampled at 32 kHz. The signal is passed through a Throttle block to regulate the processing rate.

The generated signal is divided into multiple branches using Rational Resampler blocks. Each branch demonstrates a different sampling condition including proper sampling, under-sampling, and oversampling.

Low Pass Filters are employed after resampling to suppress unwanted frequency components. QT GUI Frequency Sinks are connected to visualize the spectral behavior under each sampling condition.

Working Principle

Sampling converts a continuous-time signal into a discrete-time representation by taking measurements at uniform intervals.

When the sampling frequency satisfies the Nyquist criterion, the original signal spectrum is preserved. If the sampling frequency falls below the Nyquist rate, aliasing occurs due to spectral folding. Oversampling increases the sampling rate beyond the Nyquist requirement, improving frequency-domain representation.

System Parameters

Parameter	**Value**
Signal Frequency	12 kHz
Sample Rate	32 kHz
Oversampling Ratio	2:1
Undersampling Ratio	1:4
LPF Cutoff Frequency	14 kHz
Transition Width	1 kHz
Filter Window	Hamming

Table 2: System Parameters

Procedure

1. Open GNU Radio Companion.
2. Create a Signal Source block.
3. Set signal frequency to 12 kHz.
4. Set sample rate to 32 kHz.
5. Add a Throttle block.
6. Create separate branches using Rational Resampler blocks.
7. Configure one branch for proper sampling.
8. Configure one branch for undersampling.
9. Configure one branch for oversampling.
10. Add Low Pass Filters after each resampler.
11. Connect QT GUI Frequency Sinks.
12. Execute the flowgraph.
13. Observe spectral behavior under different sampling conditions.
14. Record the observations.

Expected Output

1. Proper sampling preserves the original spectrum.
2. Undersampling introduces aliasing.
3. Oversampling provides smoother spectral representation.
4. Frequency-domain characteristics can be clearly visualized.

Observation Table

Condition	Sampling Rate	Observation
Proper Sampling	32 kHz	Original spectrum preserved
Undersampling	8 kHz	Aliasing observed
Oversampling	64 kHz	Improved spectral resolution

Table 3: Observed Results

Result Analysis

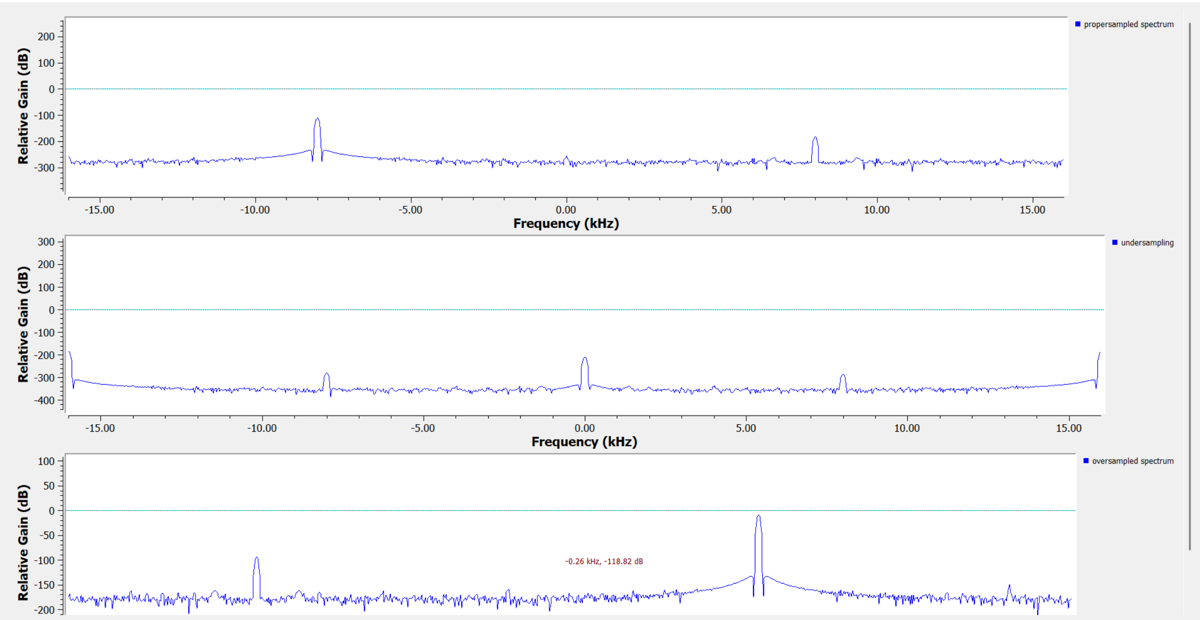

Figure 2: Comparison of Proper Sampling, Undersampling and Oversampling

The undersampled signal exhibits aliasing due to violation of the Nyquist criterion. Proper sampling accurately preserves the signal spectrum, while oversampling improves spectral smoothness and frequency resolution.

Observations

- Proper sampling preserved the original signal frequency.
- Undersampling caused spectral folding and aliasing.
- Oversampling improved spectral representation.
- Frequency-domain differences were clearly visible.

Result

Sampling techniques were successfully demonstrated using GNU Radio. Proper sampling preserved signal characteristics, undersampling produced aliasing effects, and oversampling improved spectral representation.

Applications

- Digital Signal Processing
- Software Defined Radio Systems
- Wireless Communication

- Audio Processing
- Image Processing
- Data Acquisition Systems
- Radar Systems

Precautions

1. Verify sampling frequency before execution.
2. Ensure correct interpolation and decimation values.
3. Check Low Pass Filter parameters carefully.
4. Avoid excessive CPU load.
5. Observe spectra carefully for aliasing effects.

Conclusion

The experiment successfully demonstrated the effects of proper sampling, undersampling, and oversampling using GNU Radio. The Nyquist Sampling Theorem and aliasing effects were verified through frequency-domain analysis.

References

1. GNU Radio Documentation.
2. Simon Haykin, Digital Communication Systems.
3. Proakis and Manolakis, Digital Signal Processing.
4. Software Defined Radio Principles and Applications.
5. Nyquist Sampling Theorem.